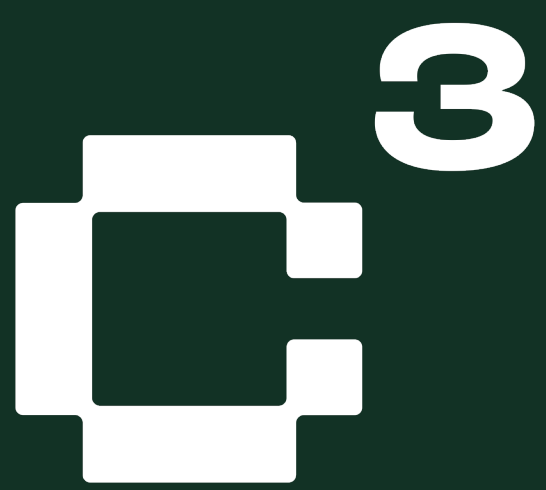




INTRODUCTION

Smart Contract Exploits Cost Billions

- A *smart contract* is a self-executing, high-level program written in Solidity that resides at a specific address on the Ethereum blockchain.
- Solidity compiles to Ethereum Virtual Machine (EVM) bytecode; once deployed, that bytecode is immutable¹.
- Exploits in these contracts have incurred total losses upwards of \$16B² in DeFi (decentralized finance).
- Additionally, Solidity presents its own unique set of challenges, and blockchain has its own native failure modes. For example, reentrancy has no analogous C/C++ vulnerability. And a missed vulnerability in Solidity code cannot be patched after deployment.
- Here, our goal is to explore *lightweight intervention* methods for a specialized machine learning classification task.

Activation Steering

- Activation steering* is an inference-time technique that modifies a Large Language Model's (LLM) internal hidden states (activations) to control its behavior without retraining or fine-tuning.
- It sits between prompting (essentially free) and fine-tuning (expensive) in terms of compute costs, since no gradient computation is required.

$$\tilde{h}^l = h^l + \alpha \cdot v^l$$

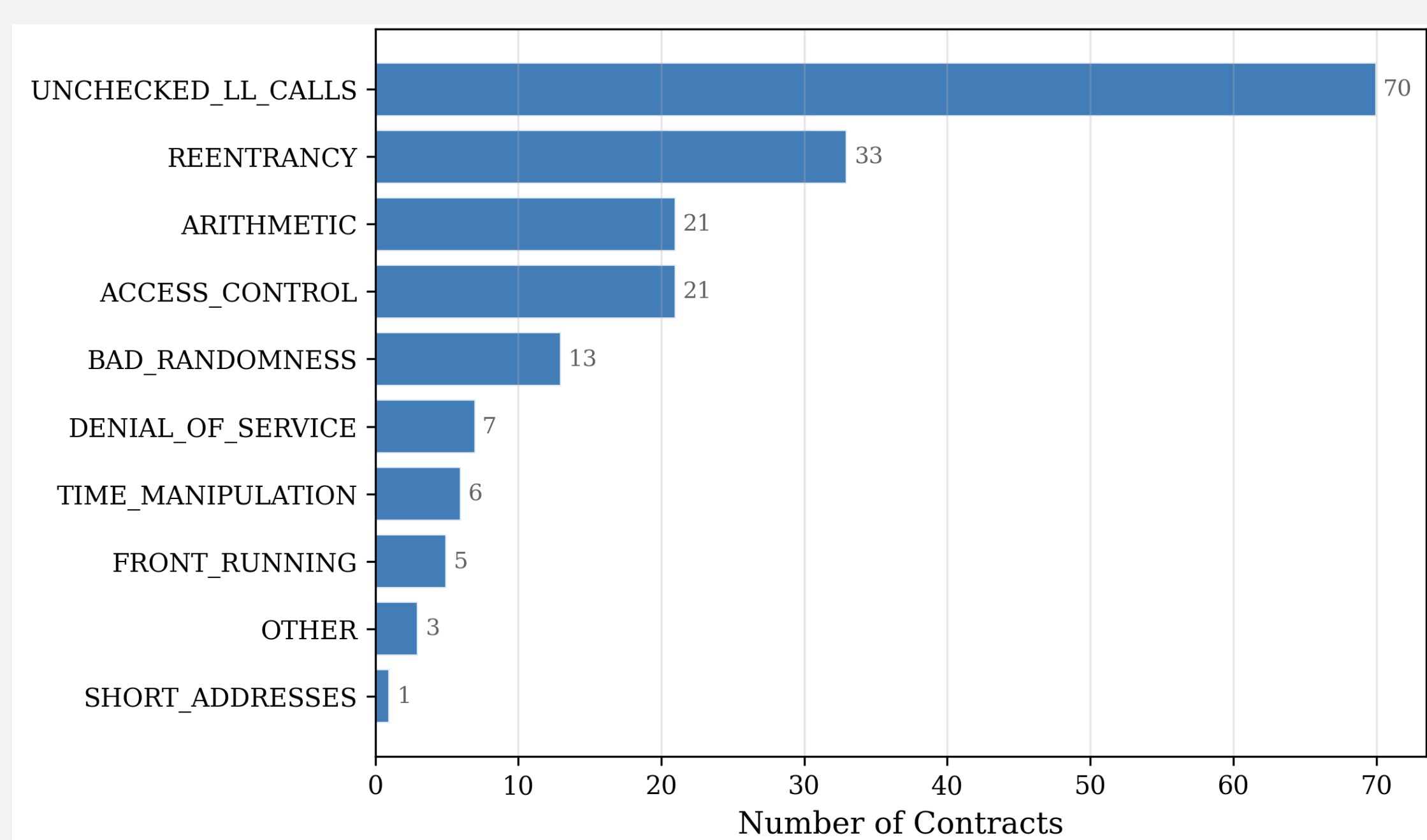
updated hidden state activation at layer l hidden state activation at layer l scalar magnitude direction vector

Approach	Examples	Strengths	Drawbacks
Static Analysis Tools	Slither ³ , Mythril, Oyente	Precise	Inflexible rules
Lightweight Prompting	Zero-shot / few-shot	Cheap, flexible	Poor performance
Fine-Tuned LLMs	VulnPatch ⁴ , Smart-LLaMA-DPO ⁵	High F1 scores	Computationally expensive

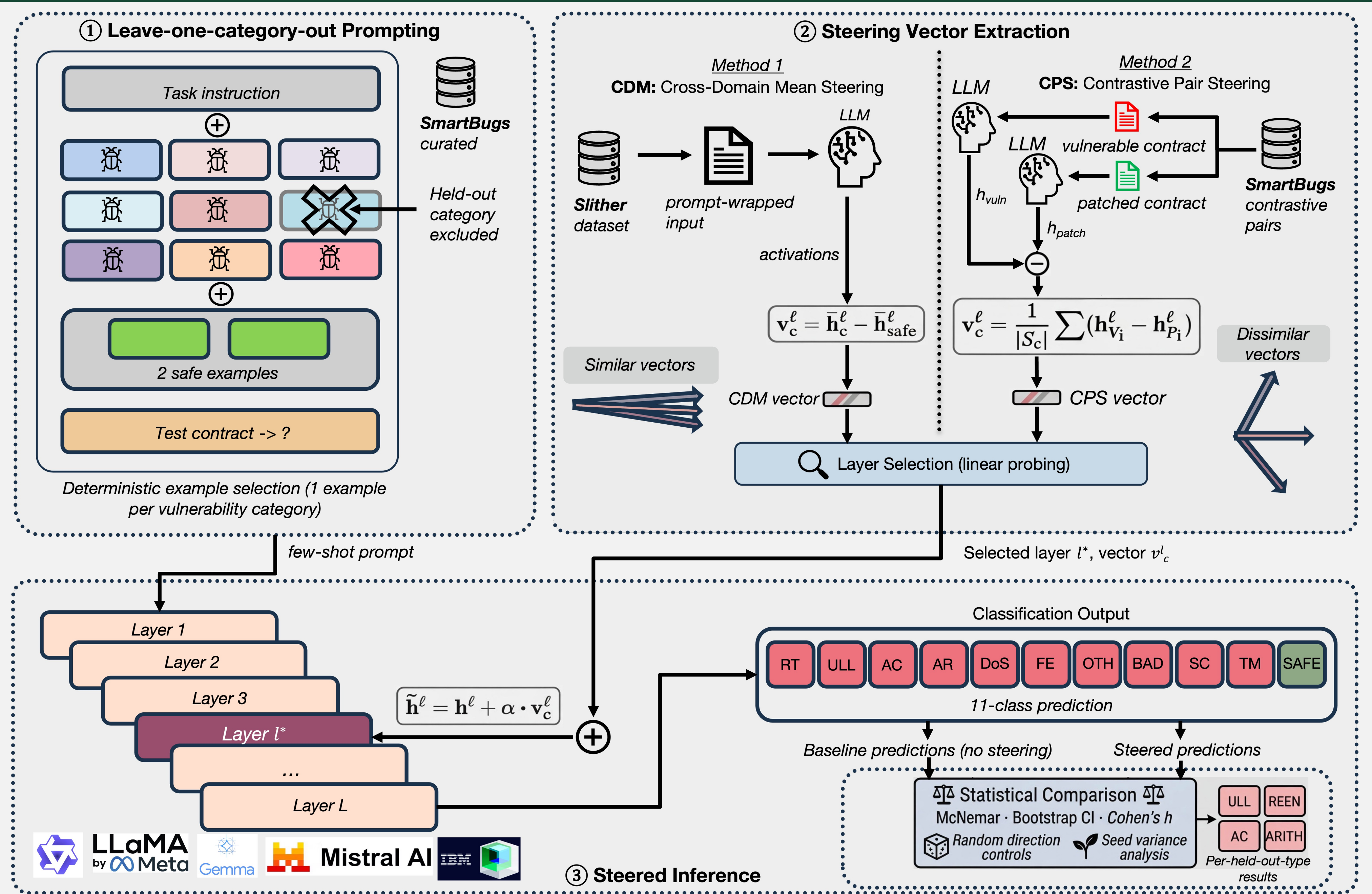
Our motivation: We are looking for interpretable, lightweight interventions that use the model's own internal representations without fine-tuning

METHODS

- Leave-one-category out Prompting:** Firstly, we construct a leave-one-out few-shot prompt.
- Steering Vector Extraction:** Next, we extract steering vectors via **CDM** (difference-in-means b/w the two groups of activations)⁶ or **CPS** (paired per-sample subtraction of activations)⁷ and select the best layer based on its AUROC curve.
- Steered Inference:** Finally, we inject the vector via a PyTorch forward hook during inference.



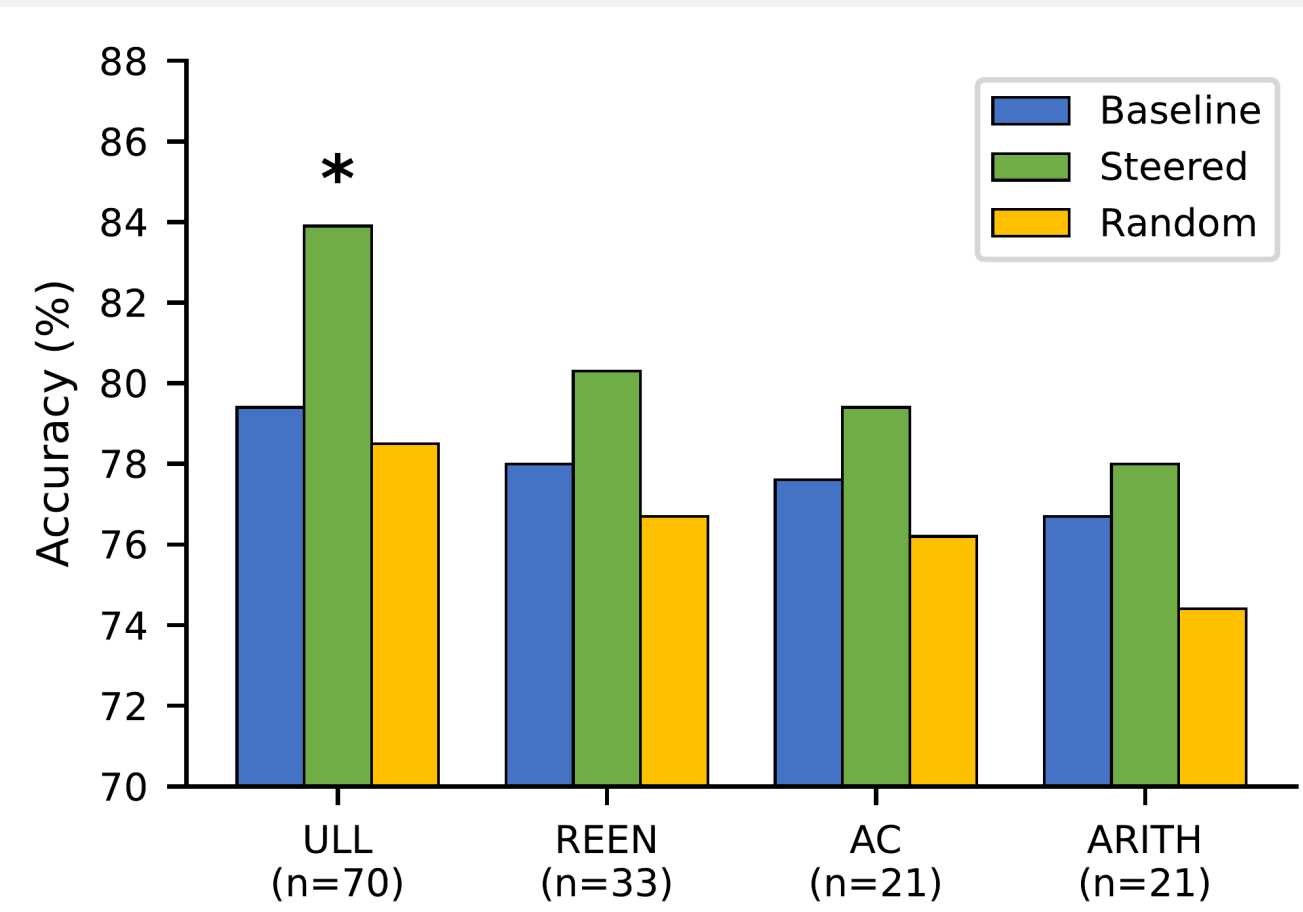
The *long-tail distribution* of our test set motivated our leave-one-category-out evaluation on the four largest vulnerability categories



RESULTS

1 Steering improves vulnerability detection

- We see a **+4.5 pp accuracy improvement** on the Unchecked Low-Level Calls (ULL) vulnerability, along with a **7% improvement in recall**.
- To rule out noise, we used random direction control vectors that have the same magnitude (same L2 norm) and are almost orthogonal to the steering direction. Naturally, such vectors should be detrimental to performance as opposed to the steering vector.



Held-out	Accuracy (%)			F1			ΔS	p-value
	Base	Steer	Rand	Base	Steer	Rand		
ULL	79.4	83.9	78.5	.855	.904	.837	+4.5	.006
REEN	78.0	80.3	76.7	.741	.811	.779	+2.2	.125
AC	77.6	79.4	76.2	.585	.579	.585	+1.8	.219
ARITH	76.7	78.0	74.4	.950	.895	.950	+1.3	.453

2 Out of six models, three show positive and significant results

Model	Params	Base	Steered	ΔS	ΔR	p
Qwen-32B	32B	79.4	83.9	+4.5	-0.9	0.006
Gemma-9B	9B	75.8	75.8	+0.0	-4.0	—
Llama-8B	8B	62.8	61.0	-1.8	+6.7	—
Granite-8B	8B	56.1	59.2	+3.1	-11.2	0.296
Mistral-7B	7B	45.3	53.4	+8.1	-13.5	0.018
CL-34B	34B	39.5	48.4	+9.0	-2.2	0.004

3 CDM produces better results than CPS

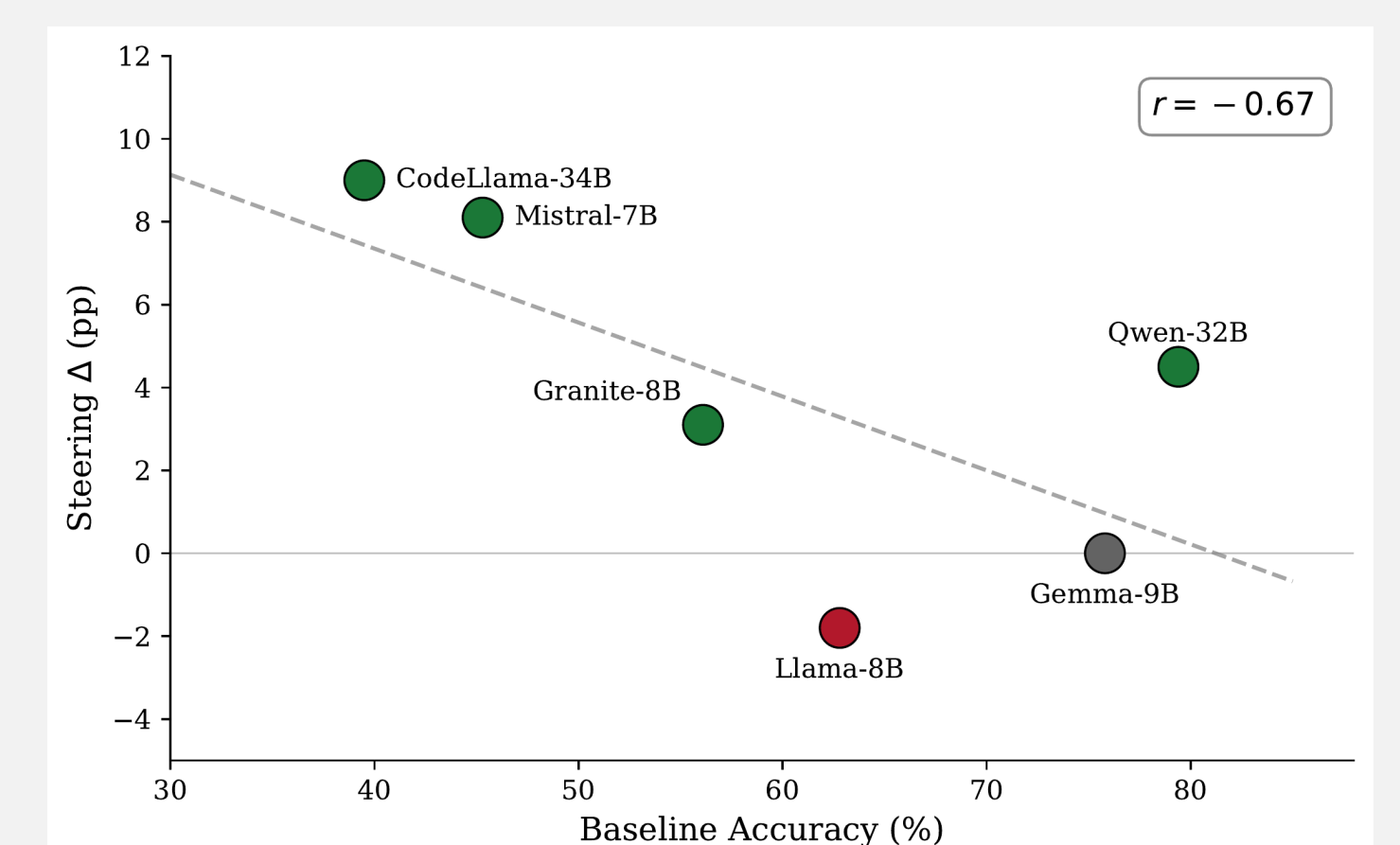
CDM (coarse)	CPS (fine)
Unpaired difference-in-means b/w the two groups of activations	Paired per-sample subtraction of activations
Category vectors nearly parallel (cos. sim. b/w 0.75-0.91)	Category vectors near-orthogonal (cos. sim. b/w 0.06-0.09)
Statistically significant results	Statistically insignificant results

- CPS is methodologically more accurate, but it performs poorly; this is consistent with our hypothesis that steering *adjusts* a decision threshold rather than adding vulnerability reasoning.
- Across all configurations, Cohen's h stays at or below 0.18 (h = 0.20 is the threshold for a small effect), which is consistent with a calibration-scale intervention.

4 A comparison between lightweight interventions

Intervention	Accuracy	Precision	F1
Zero-shot	50.7	47.9	38.8
Few-shot	79.4	69.9	62.7
Few-shot + Steer	83.9	71.1	63.5

5 Steering helps least where the model is strongest



- The Pearson correlation between baseline accuracy and steering delta is $r = -0.67$, which implies a **strong negative correlation**.
- This suggests, although not significantly at six models ($p \approx 0.14$), that stronger models benefit less from steering.
- Models close to their knowledge ceiling improve little, while weaker models see larger improvements.
- Steering activates what pre-training encodes; it does not introduce new concepts and/or reasoning.

TAKEAWAYS

Contributions

- This is the **first application** of the activation steering technique for multi-class vulnerability classification in Solidity smart contracts, with evaluation on six open-source models.
- We compare two steering vector extraction methods: cross-domain group means and domain-matched contrastive pairs, showing that steering serves to calibrate the model's confidence and not improve its reasoning.
- We systematically compare several lightweight interventions for this task: zero-shot, few-shot, and steered few-shot, and show that only steering offers improvements over few-shot prompting.
- We demonstrate that steering is directionally specific and offers improvements over noise from few-shot example selection by using random direction vectors and seed variance across six models.

Limitations and Future Work

- We tested only linear additive steering in this work. While we conclude that steering acts as a confidence threshold adjustment in linear activation space, it may yet produce stronger results under projection/gradients-based techniques.
- Additionally, the Llama-3.1-8B result indicates that directional specificity may not universally hold across all architectures.
- Although plausible, our calibration interpretation is inferred indirectly, and direct evidence such as confidence-calibration plots and confusion matrices remains future work.

References

- Upgrading smart contracts. <https://ethereum.org/developers/docs/smart-contracts/upgrading/>
- "DeFi Hacks & Exploits Database - DefiLlama," <https://defillama.com/hacks>
- J. Feist et al., "Slither: A Static Analysis Framework for Smart Contracts," IEEE/ACM WETSEB [4] Sakib, M. N. and L. Khan, "VulnPatch: Multi-Agent Automated Smart Contract Vulnerability Detection, Explanation, and Mitigation Framework," IEEE BCCA 2025
- Yu, L et al., "Smart-LLaMA-DPO: Reinforced Large Language Model for Explainable Smart Contract Vulnerability Detection," arXiv:2506.18245, 2025
- A. Zou et al., "Representation Engineering: A Top-Down Approach to AI Transparency," arXiv:2310.01405, 2023
- N. Rimsky et al., "Steering Llama 2 via Contrastive Activation Addition," ACL 2024